

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS

**DISEÑO E IMPLEMENTACIÓN DEL MÓDULO DE DASHBOARDS POR
ROL BAJO ARQUITECTURA LIMPIA PARA EL SISTEMA DE
GESTIÓN DE INVESTIGACIÓN DE LA FIIS (SGI-FIIS)**

Documento de Especificación Técnica e Implementación

Autor:

Sergio — Equipo SGI-FIIS

Curso/Proyecto:

Construcción de Software II

Tingo María, Perú

2026

Resumen

El presente documento describe el diseño, la arquitectura y el proceso de implementación del módulo de Dashboards por Rol para el Sistema de Gestión de Investigación de la Facultad de Ingeniería en Informática y Sistemas (SGI-FIIS). Este módulo constituye la vista inicial personalizada del sistema para cada usuario autenticado, proporcionando métricas institucionales, estado del flujo de trámites y alertas dinámicas adaptadas al rol de cada actor. La implementación técnica se realizó utilizando Java 17 con Spring Boot 4.1, adoptando un diseño basado en Arquitectura Limpia (Clean Architecture) para separar las reglas de negocio de la infraestructura tecnológica. La recuperación de métricas se realiza mediante consultas SQL nativas con JdbcTemplate sobre PostgreSQL 16, priorizando rendimiento en vistas agregadas de múltiples tablas relacionales. El módulo cubre los requisitos funcionales RF-88 a RF-93, además de vistas complementarias para Decano y Estudiante, e incorpora pruebas unitarias para mapper, casos de uso, controlador y repositorio, fortaleciendo la cobertura exigida por el proceso de integración continua.

Palabras clave: *Dashboard, Roles, Clean Architecture, Spring Boot, JdbcTemplate, PostgreSQL, REST API, Métricas institucionales.*

1. INTRODUCCIÓN

El desarrollo de sistemas de información universitaria para la gestión de investigación requiere interfaces adaptativas que presenten información relevante a cada actor según sus responsabilidades institucionales. Un administrador necesita una visión global del sistema, mientras que un coordinador de grupo solo debe visualizar los datos correspondientes a su grupo. Esta necesidad de personalización por rol constituye el fundamento del módulo de Dashboards por Rol del SGI-FIIS.

El módulo de Dashboards representa la primera pantalla funcional que visualiza el usuario al autenticarse en el sistema. Su objetivo fundamental es proveer, de forma inmediata y contextualizada, las métricas más relevantes para la toma de decisiones: proyectos activos, trámites pendientes, informes por atender, resoluciones emitidas y alertas de acción inmediata. La información se extrae en tiempo real desde la base de datos relacional y se entrega como respuesta JSON a través de endpoints REST diferenciados por rol.

1.1 Objetivos del Módulo

El desarrollo del módulo persigue dos objetivos centrales. Por un lado, proveer a cada rol del sistema una vista de indicadores institucionales precisa y filtrada, evitando que un usuario acceda a información que no le corresponde según su función. Por otro lado, establecer una arquitectura de consulta eficiente que soporte el crecimiento de datos del sistema sin degradación del rendimiento, mediante el uso de SQL nativo optimizado con índices estratégicos.

1.2 Justificación de la Solución

La decisión de implementar endpoints independientes por rol, en lugar de un único endpoint genérico, responde al principio de separación de responsabilidades y al requisito RF-91, que exige que el dashboard del Coordinador de Grupo muestre exclusivamente los datos de su propio grupo. Esta separación facilita además la futura integración con el módulo de autenticación, donde cada endpoint podrá protegerse con anotaciones de seguridad específicas por rol sin modificar la lógica de negocio implementada.

2. ARQUITECTURA DEL SISTEMA (CLEAN ARCHITECTURE)

Para garantizar la mantenibilidad, escalabilidad y testabilidad del módulo, se adoptó el patrón de diseño arquitectónico de Arquitectura Limpia (Clean Architecture), consistente con la arquitectura general definida para todo el proyecto SGI-FIIS. Este enfoque promueve un desacoplamiento estricto de las reglas de negocio respecto a los frameworks, bases de datos y detalles de infraestructura. La regla fundamental establece que las dependencias deben apuntar únicamente hacia adentro: el núcleo del dominio no conoce ningún detalle de las capas externas.

2.1 Estructura del Proyecto y Capas

El módulo de dashboards se divide en cuatro capas claramente delimitadas, organizadas dentro del paquete `com.sgi.fiis.dashboards` de la siguiente forma:

1. Capa de Dominio (domain/): Es el núcleo del módulo. Contiene los modelos de negocio puros por rol (DashboardAdmin, DashboardDirector, DashboardCoordinador, DashboardDocente, DashboardEvaluador, DashboardDecano, DashboardEstudiante) y la interfaz DashboardRepository que define el contrato de acceso a datos. Esta capa es completamente independiente de Spring Boot, JPA o cualquier librería de infraestructura.
2. Capa de Aplicación (application/): Contiene los Data Transfer Objects (DTOs) de respuesta y los casos de uso. Se implementa un use case por rol (ObtenerDashboardAdminUseCase, ObtenerDashboardDirectorUseCase, etc.) que orquesta la obtención de datos a través del puerto del dominio y delega la transformación al mapper de presentación.
3. Capa de Infraestructura (infrastructure/): Contiene la implementación concreta de DashboardRepository mediante DashboardRepositoryImpl, que ejecuta consultas SQL nativas a través de JdbcTemplate. Esta capa conoce los detalles de PostgreSQL pero es transparente para las capas superiores.
4. Capa de Presentación (presentation/): Contiene el controlador REST DashboardController con los siete endpoints diferenciados por rol, y el DashboardMapper responsable de transformar los modelos de dominio en DTOs de respuesta JSON.

2.2 Decisión de Diseño: JdbcTemplate vs Spring Data JPA

Se optó por JdbcTemplate con SQL nativo en lugar de repositorios Spring Data JPA porque los dashboards requieren consultas agregadas sobre múltiples tablas (COUNT, JOIN, filtros por estado). JdbcTemplate ofrece control directo sobre las queries y mejor rendimiento en este contexto de lectura agregada, alineándose con el contenido de la Unidad 4 del sílabo sobre eficiencia de desempeño y comparación ORM vs SQL directo.

2.3 Estructura de Carpetas Implementada

```
dashboards/
├── domain/
│   ├── model/           ← 7 modelos (uno por rol)
│   └── port/            ← DashboardRepository (interfaz)
├── application/
│   ├── dto/             ← 7 DTOs de respuesta JSON
│   └── usecase/         ← 7 use cases (uno por rol)
├── infrastructure/
│   └── persistence/     ← DashboardRepositoryImpl (JdbcTemplate)
├── presentation/
│   ├── controller/     ← DashboardController (7 endpoints REST)
│   └── mapper/          ← DashboardMapper (dominio → DTO)
```

3. MODELO DE DATOS Y PERSISTENCIA

La base de datos relacional PostgreSQL 16 sirve como motor de almacenamiento del backend SGI-FIIS. El módulo de dashboards no posee tablas propias; en su lugar, realiza consultas de lectura sobre las tablas definidas por los demás módulos del sistema. La base de datos fue inicializada mediante un script SQL que crea las tablas del sistema y carga los datos semilla necesarios para validar los endpoints del dashboard.

3.1 Tablas Consultadas por el Módulo

El módulo de dashboards realiza consultas de solo lectura (SELECT con COUNT y JOIN) sobre las siguientes tablas del sistema:

Tabla	Módulo propietario	Datos consultados
usuarios	[junior]	Conteo total y activos, filtro por id
roles	[junior]	Verificación de rol_principal
grupos_investigacion	[kevin]	Grupo del coordinador, conteo activos
membresias_grupo	[kevin]	Miembros activos e inactivos por grupo
proyectos	[amy]	Conteo por estado, filtro por responsable y grupo
integrantes_proyecto	[amy]	Proyectos donde el docente es integrante
tramites	[marco]	Conteo por estado y rol_revisor_actual
informes_avance	[jorge]	Conteo de informes pendientes por proyecto
evaluaciones	[dairon]	Evaluaciones asignadas, pendientes y resultados
resoluciones	[sebas]	Conteo total de resoluciones emitidas
convocatorias	[amy]	Convocatorias en estado ABIERTA
planes_tesis	[por asignar]	Estado del plan del estudiante y conteo por grupo
documentos	[Ale]	Conteo de documentos por usuario

Tabla 1. Tablas del sistema consultadas por el módulo de dashboards.

3.2 Índices Estratégicos

El script de inicialización de la base de datos incluye índices estratégicos sobre las columnas más utilizadas en las cláusulas WHERE de las consultas del dashboard. A continuación se presentan los índices relevantes para el módulo:

```
-- Índices que benefician las consultas de dashboards
CREATE INDEX ix_usuarios_rol      ON usuarios(id_rol_principal);
CREATE INDEX ix_usuarios_activo  ON usuarios(es_activo);
CREATE INDEX ix_proyectos_responsable ON proyectos(id_responsable);
```

```
CREATE INDEX ix_proyectos_grupo      ON proyectos(id_grupo);  
CREATE INDEX ix_proyectos_estado     ON proyectos(estado_proyecto);  
CREATE INDEX ix_tramites_estado      ON tramites(estado_actual, rol_revisor_actual);  
CREATE INDEX ix_tramites_grupo       ON tramites(id_grupo);  
CREATE INDEX ix_tramites_solicitante ON tramites(id_solicitante);  
CREATE INDEX ix_informes_proyecto    ON informes_avance(id_proyecto);  
CREATE INDEX ix_evaluaciones_evaluador ON evaluaciones(id_evaluador);
```

Listado 1. Índices estratégicos de la base de datos que optimizan las consultas de dashboards.

4. ESPECIFICACIÓN FUNCIONAL Y REQUISITOS CUBIERTOS

El módulo de Dashboards por Rol cubre los requisitos funcionales RF-88 a RF-93 del documento de especificación del sistema SGI-FIIS, correspondientes al módulo 3.13 Dashboards y Reportes. La siguiente tabla presenta el estado de implementación de cada requisito:

Código	Requisito Funcional	Prioridad	Estado
RF-88	El sistema debe mostrar un dashboard general para el Administrador.	Alta	Implementado
RF-89	El sistema debe mostrar un dashboard institucional para el Director de Investigación.	Alta	Implementado
RF-90	El sistema debe mostrar un dashboard específico para cada Coordinador de Grupo.	Alta	Implementado
RF-91	El dashboard del Coordinador debe mostrar solo miembros, proyectos, trámites e informes de su grupo.	Alta	Implementado
RF-92	El sistema debe mostrar un dashboard para el Docente Investigador con sus proyectos, documentos, trámites e informes.	Alta	Implementado
RF-93	El sistema debe mostrar un dashboard para el Evaluador con proyectos y evaluaciones asignadas.	Media	Implementado
RF-03	El sistema debe redirigir al usuario a su dashboard según su rol al iniciar sesión.	Alta	Pend. módulo auth
RF-23	El dashboard del Coordinador se actualiza automáticamente según el grupo al que pertenece.	Alta	Implementado

Tabla 2. Trazabilidad de requisitos funcionales cubiertos por el módulo de dashboards.

4.1 Endpoints REST Implementados

Todos los endpoints se encuentran bajo el prefijo base /api/dashboard. El parámetro {idUserario} identifica al usuario autenticado; cuando el módulo de autenticación esté disponible, este parámetro será reemplazado por el contexto de seguridad de Spring Security.

Método	Endpoint	Rol	RF Cubierto
GET	/api/dashboard/admin/{idUserario}	ADMIN	RF-88
GET	/api/dashboard/director/{idUserario}	DIRECTOR_INVESTIGACION	RF-89
GET	/api/dashboard/coordinador/{idUserario}	COORDINADOR_GRUPO	RF-90, RF-91, RF-23
GET	/api/dashboard/docente/{idUserario}	DOCENTE_INVESTIGADOR	RF-92
GET	/api/dashboard/evaluador/{idUserario}	EVALUADOR	RF-93
GET	/api/dashboard/decano/{idUserario}	DECANO	Flujo institucional
GET	/api/dashboard/estudiante/{idUserario}	ESTUDIANTE	RN-06

Tabla 3. Endpoints REST implementados en el módulo de dashboards.

5. DETALLES DE IMPLEMENTACIÓN TÉCNICA

La materialización del módulo en Spring Boot se rige por la estructuración de componentes desacoplados que operan en conjunto bajo el patrón de Arquitectura Limpia. A continuación se describen los componentes técnicos más relevantes de cada capa.

5.1 Modelos de Dominio

Cada rol cuenta con un modelo de dominio independiente anotado con `@Builder` y `@Getter` de Lombok. Los modelos incluyen una lista de alertas dinámicas (`AlertaItem`) generadas en la capa de infraestructura según el estado actual de la base de datos. El siguiente extracto corresponde al modelo del Coordinador de Grupo, que ilustra la separación entre datos del grupo y métricas de trámites:

```
@Getter @Builder
public class DashboardCoordinador {
    // Identificación del grupo (RF-90)
    private int idGrupo;
    private String nombreGrupo;
    private String codigoGrupo;

    // Métricas exclusivas del grupo (RF-91)
    private int totalMiembros;
    private int miembrosActivos;
    private int totalProyectosGrupo;
    private int proyectosActivosGrupo;
    private int tramitesPendientesGrupo;
    private int informesAvanceGrupo;
    private int planesTesisGrupo;

    // Flujo de trámites del grupo
    private int tramitesPostulados;
    private int tramitesEnRevision;
    private int tramitesAprobados;
    private int tramitesObservados;

    private List<AlertaItem> alertas;

    @Getter @Builder
    public static class AlertaItem {
        private String tipo; // ALERTA | REVISION | INFO
        private String titulo;
        private String descripcion;
    }
}
```

Listado 2. Modelo de dominio DashboardCoordinador con métricas filtradas por grupo.

5.2 Puerto del Dominio

La interfaz `DashboardRepository` define el contrato de acceso a datos del módulo. Esta interfaz reside en la capa de dominio y es implementada por la capa de infraestructura, aplicando el principio de inversión de dependencias. Los casos de uso consumen exclusivamente esta interfaz, sin conocer ningún detalle de la implementación concreta.

```
public interface DashboardRepository {
    DashboardAdmin obtenerDashboardAdmin(Integer idUsuario);
    DashboardDirector obtenerDashboardDirector(Integer idUsuario);
    DashboardCoordinador obtenerDashboardCoordinador(Integer idUsuario);
    DashboardDocente obtenerDashboardDocente(Integer idUsuario);
    DashboardEvaluador obtenerDashboardEvaluador(Integer idUsuario);
    DashboardDecano obtenerDashboardDecano(Integer idUsuario);
    DashboardEstudiante obtenerDashboardEstudiante(Integer idUsuario);
}
```

Listado 3. Puerto (interfaz) del repositorio de dashboards en la capa de dominio.

5.3 Casos de Uso

Se implementa un caso de uso por rol, siguiendo el principio de responsabilidad única. Cada caso de uso recibe el identificador del usuario, delega la obtención de datos al repositorio y la transformación al mapper. El siguiente extracto corresponde al caso de uso del Administrador:

```
@Service
@RequiredArgsConstructor
public class ObtenerDashboardAdminUseCase {

    private final DashboardRepository dashboardRepository;
    private final DashboardMapper dashboardMapper;

    public DashboardAdminResponse ejecutar(Integer idUsuario) {
        DashboardAdmin modelo = dashboardRepository
            .obtenerDashboardAdmin(idUsuario);
        return dashboardMapper.toAdminResponse(modelo);
    }
}
```

Listado 4. Caso de uso ObtenerDashboardAdminUseCase en la capa de aplicación.

5.4 Implementación del Repositorio con JdbcTemplate

La clase `DashboardRepositoryImpl` implementa el puerto del dominio utilizando `JdbcTemplate` para ejecutar consultas SQL nativas. Se utiliza un método auxiliar `count()` que

ejecuta consultas `SELECT COUNT(*)` de forma segura, manejando valores nulos. Además, las consultas repetidas, filtros comunes y tipos de alerta se centralizan en constantes privadas para reducir duplicación, mejorar mantenibilidad y cumplir con las reglas de calidad del análisis estático.

```
@Repository
@RequiredArgsConstructor
public class DashboardRepositoryImpl implements DashboardRepository {

    private static final String TIPO_REVISION = "REVISION";
    private static final String SQL_CONVOCATORIAS_ABIERTAS =
        "SELECT COUNT(*) FROM convocatorias "
        + "WHERE estado = 'ABIERTA'";
    private static final String FILTRO_NO_CERRADO =
        " AND estado_actual NOT IN ('APROBADO', 'RECHAZADO')";

    private final JdbcTemplate jdbcTemplate;

    @Override
    public DashboardDirector obtenerDashboardDirector(Integer idUsuario) {
        int pendientes = count("SELECT COUNT(*) FROM tramites WHERE "
            + "rol_revisor_actual = 'DIRECTOR_INVESTIGACION'"
            + FILTRO_NO_CERRADO);
        int convocatorias = count(SQL_CONVOCATORIAS_ABIERTAS);
        // ... más métricas y alertas dinámicas ...

        return DashboardDirector.builder()
            .tramitesPendientesRevision(pendientes)
            .convocatoriasAbiertas(convocatorias)
            .alertas(alertas)
            .build();
    }

    private int count(String sql) { /* queryForObject seguro */ }
}
```

Listado 5. Fragmento de `DashboardRepositoryImpl` con consultas SQL nativas.

5.5 Controlador REST

El `DashboardController` expone siete endpoints REST bajo el prefijo `/api/dashboard`. Cada endpoint delega la solicitud al caso de uso correspondiente y retorna un DTO de respuesta con `ResponseEntity.ok()`. Actualmente el identificador del usuario se recibe mediante `@PathVariable`; cuando se integre el módulo de autenticación, este valor deberá obtenerse desde el contexto de seguridad.

```
@RestController
@RequestMapping("/api/dashboard")
```

```
@RequiredArgsConstructor
public class DashboardController {

    private final ObtenerDashboardAdminUseCase dashboardAdminUseCase;
    private final ObtenerDashboardDirectorUseCase dashboardDirectorUseCase;
    private final ObtenerDashboardCoordinadorUseCase dashboardCoordinadorUseCase;
    // ... demás use cases ...

    /** RF-88: Dashboard para el Administrador del Sistema. */
    @GetMapping("/admin/{idUser}")
    public ResponseEntity<DashboardAdminResponse> getDashboardAdmin(
        @PathVariable Integer idUsuario) {
        return ResponseEntity.ok(dashboardAdminUseCase.ejecutar(idUsuario));
    }

    /** RF-90, RF-91: Dashboard para Coordinador - solo su grupo. */
    @GetMapping("/coordinador/{idUser}")
    public ResponseEntity<DashboardCoordinadorResponse> getDashboardCoordinador(
        @PathVariable Integer idUsuario) {
        return ResponseEntity.ok(dashboardCoordinadorUseCase.ejecutar(idUsuario));
    }
    // ... demás endpoints ...
}
```

Listado 6. Fragmento del DashboardController con endpoints REST por rol.

6. SISTEMA DE ALERTAS DINÁMICAS

Cada dashboard incluye una lista dinámica de alertas generadas en la capa de infraestructura según el estado actual de la base de datos. Las alertas se clasifican en tres tipos según su nivel de urgencia:

Tipo	Significado	Ejemplo de uso
ALERTA	Requiere atención inmediata	Proyectos observados, informes vencidos
REVISION	Elemento en proceso de revisión	Trámites pendientes de atender
INFO	Información relevante sin urgencia	Convocatoria activa disponible

Tabla 4. Tipos de alertas del sistema de dashboards y su significado.

Las alertas se generan condicionalmente: solo se incluyen en la respuesta si la condición que las origina es verdadera en ese momento. Por ejemplo, la alerta de proyectos observados solo se incluye si existe al menos un proyecto con estado OBSERVADO en la base de datos. Esta lógica reside en `DashboardRepositoryImpl`, manteniendo los use cases limpios de condicionales de presentación.

7. VERIFICACIÓN FUNCIONAL

El módulo fue verificado funcionalmente con la base de datos inicializada con los datos semilla del script SQL. Se realizaron pruebas manuales con Postman sobre el endpoint del Administrador, obteniendo la siguiente respuesta con estado HTTP 200 OK:

```
GET http://localhost:8080/api/dashboard/admin/1

HTTP/1.1 200 OK
Content-Type: application/json

{
  "totalUsuarios": 0,
  "totalUsuariosActivos": 0,
  "totalGrupos": 7,
  "totalGruposActivos": 7,
  "totalProyectos": 0,
  "proyectosActivos": 0,
  "tramitesPendientes": 0,
  "resolucionesEmitidas": 0,
  "tramitesEnRevision": 0,
  "tramitesAprobados": 0,
  "tramitesRechazados": 0,
  "alertas": []
}
```

Listado 7. Respuesta del endpoint /api/dashboard/admin/1 con datos semilla.

El valor totalGrupos: 7 confirma que los datos semilla (GINSOFT, RESEGTI, GISI, CICO, EAP, MAP, EU) fueron cargados correctamente. Los demás campos en cero son el comportamiento esperado dado que la base de datos no contiene aún usuarios ni proyectos creados por los demás módulos del equipo.

Verificación	Resultado esperado	Estado
Conexión a PostgreSQL	CONNECTED	Verificado
GET /api/dashboard/admin/1	HTTP 200 + JSON válido	Verificado
totalGrupos en respuesta	7 (datos semilla)	Verificado
totalGruposActivos en respuesta	7	Verificado
Respuesta JSON sin errores 500	Todos los endpoints	Verificado
Pruebas unitarias Maven	mvn test sin errores	Verificado
GitHub Actions - Backend CI	Build y test exitosos	Verificado
Análisis estático SonarCloud	Cobertura y duplicación monitoreadas	En seguimiento

Tabla 5. Resultados de verificación funcional manual y automatizada del módulo de dashboards.

7.2 Verificación automatizada y control de calidad

Además de la verificación manual con Postman, el módulo cuenta con pruebas automatizadas ejecutadas por Maven y validadas en el flujo de integración continua. Estas pruebas no dependen de una base de datos externa para el módulo de dashboards, ya que se emplean mocks sobre JdbcTemplate, casos de uso y servicios de aplicación.

- `DashboardMapperTest`: valida el mapeo de los siete modelos de dominio hacia sus DTOs, incluyendo el caso de listas de alertas nulas.
- `ObtenerDashboardUseCaseTest`: verifica que cada caso de uso invoque el método correcto del puerto `DashboardRepository` y delegue la conversión al `DashboardMapper`.
- `DashboardControllerTest`: prueba los siete endpoints REST con `MockMvc` en modo `standaloneSetup`, evitando dependencias innecesarias del `ApplicationContext`.
- `DashboardRepositoryImplTest`: valida la construcción de métricas y alertas usando `JdbcTemplate` mockeado, incluyendo escenarios con y sin grupo asignado.

8. NOTAS DE INTEGRACIÓN Y MEJORAS FUTURAS

8.1 Integración con Módulo de Autenticación

Cuando el módulo auth [junior] complete la implementación del sistema de autenticación con JWT, se deberán realizar los siguientes ajustes en el módulo de dashboards:

- Reemplazar `@PathVariable Integer idUsuario` por el objeto `Authentication` de Spring Security en `DashboardController`, extrayendo el id del usuario desde el token JWT o la sesión.
- Agregar anotaciones `@PreAuthorize("hasRole('ADMIN')")` o similares en cada endpoint para protegerlos según el rol requerido.
- Implementar RF-03: la respuesta del endpoint de login debe incluir la URL del dashboard correspondiente al rol del usuario autenticado.

8.2 Pruebas unitarias implementadas

Como parte de los requisitos del curso (Unidad 2 - Pruebas Unitarias), se implementaron pruebas con JUnit 5 y Mockito para cubrir las capas principales del módulo. La estrategia prioriza pruebas unitarias aisladas y estables para el pipeline de CI, evitando depender de una base de datos real cuando el objetivo es validar la lógica del dashboard.

- Use cases de dashboards: verifican que cada caso de uso invoque el método correcto del repositorio y transforme el modelo mediante DashboardMapper.
- DashboardMapper: valida el mapeo completo de campos y alertas para Administrador, Director, Coordinador, Docente, Evaluador, Decano y Estudiante.
- DashboardControllerTest: valida las rutas REST y la serialización JSON de los siete endpoints usando MockMvc standalone.
- DashboardRepositoryImplTest: cubre métricas y alertas mediante JdbcTemplate mockeado, incluyendo flujos de coordinador con grupo, coordinador sin grupo y estudiante con plan/grupo.

8.3 Mejoras Futuras

- Implementar caché en métricas de alta frecuencia para mejorar el rendimiento (RNF-17).
- Agregar pruebas de carga con k6 o Locust sobre los endpoints de dashboards para validar el comportamiento bajo concurrencia (Unidad 4 del sílabo).
- Incorporar paginación en la lista de alertas si el volumen de datos crece significativamente.

9. CONCLUSIONES Y RECOMENDACIONES

La construcción e implementación del módulo de Dashboards por Rol para el SGI-FIIS aporta las siguientes conclusiones:

- La adopción de la Arquitectura Limpia (Clean Architecture) reduce sustancialmente el acoplamiento del núcleo de la aplicación. La separación entre dominio, aplicación, infraestructura y presentación permite modificar cualquier capa de forma independiente sin afectar las demás. En particular, la futura integración del módulo de autenticación solo requiere modificaciones en la capa de presentación, sin tocar los use cases ni las queries SQL.
- El uso de JdbcTemplate con SQL nativo en lugar de Spring Data JPA es la decisión técnica correcta para el caso de dashboards: las consultas son de solo lectura, agregan datos de múltiples tablas con COUNT y JOIN, y requieren control preciso sobre el SQL generado. Esta decisión se alinea con el contenido de la Unidad 4 del sílabo sobre eficiencia de desempeño y comparación ORM vs SQL directo.
- El diseño de endpoints independientes por rol, en lugar de un único endpoint genérico, cumple estrictamente con el requisito RF-91 y facilita la aplicación futura de autorización por rol sin modificar la lógica de negocio implementada.
- La implementación de alertas dinámicas generadas condicionalmente desde la capa de infraestructura agrega valor funcional al módulo, permitiendo al usuario visualizar el estado real del sistema sin necesidad de navegar por múltiples pantallas.

Se recomienda mantener la cobertura de pruebas unitarias del módulo como parte del flujo de CI, complementar la validación con pruebas de carga (k6 o Locust) y continuar monitoreando duplicación y cobertura mediante SonarCloud. La futura integración con autenticación deberá incorporar pruebas específicas de autorización por rol para asegurar que cada endpoint solo sea consumido por los actores correspondientes.

REFERENCIAS

- Martin, R. C. (2018). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Spring Projects. (2026). Spring Framework Documentation — JdbcTemplate. VMware Tanzu.
<https://docs.spring.io/spring-framework/reference/data-access/jdbc.html>
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- SonarSource. (2026). SonarQube Cloud Documentation.
<https://docs.sonarsource.com/sonarqube-cloud/>